

LAPORAN TUGAS KECIL

IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Ice Sliding Puzzle Solver

Dipersiapkan oleh:

Benedict Darrel Setiawan – 13524057

**Sekolah Teknik Elektro dan Informatika - Institut Teknologi
Bandung**

Jl. Ganesha 10, Bandung 40132

1. Pendahuluan

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

Fungsi dari program yang dirancang adalah untuk mencari sebuah solusi dari sebuah puzzle Ice Sliding dengan menggunakan aplikasi route/path planning.

2. Penjelasan Algoritma

2.1 Algoritma yang Digunakan

Di dalam program Ice Sliding Puzzle solver ini digunakan tiga algoritma pencarian yaitu,

a. Uniform Cost Search (UCS)

UCS merupakan sebuah algoritma pencarian graf yang akan mengekskansi node yang memiliki nilai cost $g(n)$ paling rendah. Algoritma ini akan menjamin menghasilkan rute terpendek karena pencarian dilakukan berdasarkan nilai cost aktual dan bukan prediksi. Namun, algoritma ini memiliki kompleksitas waktu dan ruang yang cukup besar yaitu $O(b^{(1 + C^*/\epsilon)})$ dengan b merupakan banyaknya suksesor per node, C^* merupakan nilai cost dari solusi optimal, dan ϵ merupakan minimum cost per langkah. Berikut adalah langkah-langkah dari algoritma UCS yang diimplementasikan di program ini:

1. Tambah start node ke priority queue
2. Pilih node dengan nilai $g(n)$ terendah dari priority queue dan ekspansi node tersebut dengan memasukan seluruh node yang bertetangga dengannya ke priority queue.
3. Jika terdapat sebuah node bertetangga yang sudah ada priority queue, cek apakah total cost dari node di priority queue lebih tinggi daripada yang diluar. Jika iya, update node tersebut menjadi nilai total cost yang lebih rendah.
4. Cek apakah node yang baru saja diekspansi merupakan goal node. Jika iya maka algoritma selesai. Jika tidak maka algoritma tetap berjalan hingga ditemukan goal node atau priority queue kosong.

b. Greedy Best First Search (GBFS)

GBFS merupakan sebuah algoritma pencarian graf yang mengekskansi node yang memiliki nilai heuristik $h(n)$ yang paling rendah. Algoritma ini tidak menjamin rute terpendek karena pencarian dilakukan berdasarkan prediksi nilai heuristic. Namun algoritma ini memiliki kompleksitas waktu dan ruang yang lebih rendah dari kebanyakan algoritma pencarian lainnya yaitu $O(b^m)$ dengan b merupakan banyaknya suksesor per node, dan m merupakan kedalaman maksimum graf. Berikut adalah langkah-langkah dari algoritma GBFS yang diimplementasikan di program ini:

1. Tambah start node ke priority queue

2. Pilih node dengan nilai $h(n)$ terendah dari priority queue dan ekspansi node tersebut dengan memasukan seluruh node yang bertetangga dengannya ke priority queue.
3. Cek apakah node yang baru saja diekspansi merupakan goal node. Jika iya maka algoritma selesai. Jika tidak maka algoritma tetap berjalan hingga ditemukan goal node atau priority queue kosong.

c. A*

Algoritma A* merupakan algoritma pencarian graf yang mengekspansi node berdasarkan nilai evaluasi $f(n) = g(n) + h(n)$ yang paling rendah. Jika menggunakan heuristik yang admissible, maka algoritma ini akan selalu menghasilkan rute terpendek. Algoritma A* memiliki kompleksitas waktu worst-case $O(b^d)$ dengan b merupakan banyaknya suksesor per node, dan d merupakan panjang rute terpendek dari start node ke goal node. Berikut adalah langkah-langkah dari algoritma A* yang diimplementasikan di program ini:

1. Tambah start node ke priority queue
2. Pilih node dengan nilai $f(n)$ terendah dari priority queue dan ekspansi node tersebut dengan memasukan seluruh node yang bertetangga dengannya ke priority queue.
3. Jika terdapat sebuah node bertetangga yang sudah ada priority queue, cek apakah total cost dari node di priority queue lebih tinggi daripada yang diluar. Jika iya, update node tersebut menjadi nilai total cost yang lebih rendah.
4. Jika node belum dimasukan, dan terdapat sebuah node bertetangga yang sudah ada di closed list, cek apakah total cost dari node di closed list lebih tinggi daripada yang tidak. Jika iya, masukan node ke dalam priority queue.
5. Cek apakah node yang baru saja diekspansi merupakan goal node. Jika iya maka algoritma selesai. Jika tidak maka algoritma tetap berjalan hingga ditemukan rute terpendek atau priority queue kosong.
6. Masukan node yang diekspansi ke dalam closed list

2.2 Heuristik yang Digunakan

a. *Closest To Checkpoint by Coordinate Manhattan Distance*

Heuristik ini ditentukan berdasarkan Manhattan Distance dari sebuah node ke node checkpoint berikutnya. Jika tidak ada node checkpoint di puzzle atau semua node checkpoint sudah dikunjungi, maka node berikutnya yang dibandingkan berikutnya adalah goal node. Heuristik ini tidak admissible karena akan melakukan overestimasi jika semua cost tile di board sama dengan 0,

b. *Closest To Checkpoint by Cost Manhattan Distance*

Heuristik ini hampir sama dengan *Closest To Checkpoint by Coordinate Manhattan Distance*, yaitu menghitung nilai berdasarkan Manhattan Distance. Namun heuristik ini menghitung cost dari rute minimal dengan pergerakan di sumbu x dan sumbu y menuju ke node checkpoint terdekat. Heuristik ini merupakan heuristik yang admissible dikarenakan menghitung cost aktual secara langsung sehingga tidak akan terjadi overestimasi.

c. *Checkpoints Away From Goal*

Heuristik ini menghitung nilai berdasarkan banyak checkpoint yang perlu dikunjungi sebelum dapat masuk ke goal dikalikan dengan rata-rata cost dari setiap edge di graf. Heuristik ini tidak admissible karena akan melakukan overestimasi jika terdapat sebuah edge yang memiliki cost outlier.

3. Pengujian Program

Input		Output
data/test.txt Uniform Cost Search		
data/test2.txt Greedy Best First Search Closest To Checkpoint by Coordinate Manhattan Distance		
data/test2.txt A* Closest To Checkpoint by Cost Manhattan Distance		
data/test3.txt A* Closest To Checkpoint by Cost Manhattan Distance		
data/test3.txt Greedy Best First Search Checkpoints Away from Goal		

4. Lampiran

Repository proyek : <https://github.com/BenedictD-RH/Tucil3> 13524057

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

5. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Benedict Darrel Setiawan